

TaxiLink — Plan mobile complet

D'abord expliqué à un enfant de 10 ans, puis avec le vocabulaire technique (chaque mot expliqué)

Aujourd'hui, TaxiLink est un site web qui marche bien (les patrons et chauffeurs s'en servent déjà). Mais on veut aller plus loin : transformer ce site en **3 vraies applications** qu'on installe sur l'iPhone et l'Android, comme Uber.

Ce document explique **tout le plan** de cette transformation, en deux niveaux : d'abord avec des images simples (pour comprendre vite), puis avec le vocabulaire des développeurs (pour ceux qui veulent les détails).

État au 10 mai 2026 Phase 0 (la boîte à outils partagée) **terminée en 1 jour**. Reste 4 phases.



Sommaire

#	Section	Niveau
1	Les 3 voitures télécommandées	Enfant 10 ans
2	Phase 0 — La boîte à outils partagée (☒ fait)	Enfant 10 ans
3	Phase 1 — La voiture du chauffeur (12 étapes)	Enfant 10 ans
4	Phase 2 — La voiture du patron (5 étapes)	Enfant 10 ans
5	Phase 3 — La voiture du patient/famille (5 étapes)	Enfant 10 ans
6	Phase 4 — Mise dans les magasins d'apps (3 étapes)	Enfant 10 ans
7	La stack technique en 5 minutes	Technique
8	Phase 0 technique — Refactor monorepo	Technique
9	Phase 1 technique — App chauffeur (semaine par semaine)	Technique
10	Phase 2/3/4 technique	Technique
11	Récap d'avancement	Mixte

Partie 1 — Pour un enfant de 10 ans

Imagine, dessine, comprends — sans aucun mot compliqué

1. Les 3 voitures télécommandées

Imagine que TaxiLink, c'est comme construire **3 voitures télécommandées**. Chacune est faite pour une personne différente :



App Chauffeur

Pour ceux qui conduisent.
Voient les courses, acceptent,
suivent le trajet, parlent au
micro.



App Patron

Pour les chefs d'entreprise de
taxis. Suivent leurs employés,
gèrent l'agenda.



App Patient/Famille

Pour les patients âgés et leurs
enfants. Voient le taxi arriver,
l'historique, les factures.

L'image facile à comprendre

On a déjà construit une **maquette en carton** (le site web qui marche déjà). Maintenant, on construit les **vraies voitures avec moteur** — celles qu'on peut installer sur ton téléphone.

2. Phase 0 — La boîte à outils partagée ✓

✓ Ce qu'on a fait aujourd'hui (10 mai 2026)

On a fabriqué une **grande boîte à outils**. Dedans, il y a **tous les boulons, vis et roues** qui serviront aux 3 voitures. Avant, ces outils étaient cachés dans le carton du site web. Maintenant, ils sont rangés dans une boîte à part que tout le monde peut prendre.

C'est très important : sans cette boîte, on devrait **refabriquer chaque boulon trois fois**. Avec la boîte, on les fabrique une seule fois.

3. Phase 1 — La voiture du chauffeur (12 étapes) 🚗

C'est la voiture la plus importante, alors on commence par elle. Voici les **12 étapes** :

1 On allume la voiture pour la première fois

On crée une appli vide qui dit juste « Bonjour TaxiLink ». On la met sur ton téléphone Android pour vérifier qu'elle s'allume bien. C'est comme tourner la clé dans le contact pour voir si le moteur démarre.

2 Le chauffeur peut entrer dans la voiture

On ajoute la **page de connexion** : email + mot de passe, ou Google. Une fois connecté, l'appli se souvient de lui pour toujours, comme Instagram.

3 Le chauffeur voit la liste des courses disponibles

Comme Uber côté chauffeur : une **liste de courses** qu'il peut prendre. Si quelqu'un poste une nouvelle course, elle apparaît tout de suite, sans qu'il ait besoin de rafraîchir. C'est de la *magie en direct*.

4 Le chauffeur peut accepter une course

Il appuie sur une course, il voit une **carte** avec le point de départ et d'arrivée, le trajet en bleu. Il appuie sur « Accepter ». Pouf, c'est à lui.

5 La voiture sait où elle est

On allume le **GPS** : tant que l'appli est ouverte, le téléphone envoie sa position au serveur toutes les 5 secondes. Comme ça, le patron voit où sont ses chauffeurs sur sa carte.

6 Le GPS marche même téléphone éteint 🤖 (la plus dure)

C'est l'étape compliquée. Normalement, quand tu fermes une appli sur ton tel, elle dort. Mais nous, on veut que le GPS continue d'envoyer la position **même quand le chauffeur a mis son tel dans sa poche écran éteint**, sinon on perd la trace pendant la course. Apple est très strict là-dessus, il faut bien expliquer pourquoi on en a besoin (c'est pour la facturation CPAM et pour rassurer le patient).

7 Les notifications « ding ! » sur le téléphone

Quand un patron poste une nouvelle course, **tous les chauffeurs en ligne du département** reçoivent un « ding ! » sur leur tel, comme un SMS. Ils tapent dessus, ça ouvre directement la course. Pas besoin que l'appli soit ouverte.

8 Mode course active 🎮

Le chauffeur a accepté une course. L'appli passe en **mode plein écran** avec : la carte qui montre où il est, le temps restant pour arriver, l'écran qui reste allumé tout du long. Quand il arrive au point d'arrivée, l'appli **détecte automatiquement** qu'il est arrivé et passe à l'étape suivante. Magique.

9 La voix : « Salut, course de Marseille à Aubagne » 🎤

Le chauffeur appuie sur le **bouton volume du téléphone** et parle. L'ordinateur écoute, comprend ce qu'il dit, et crée la course toute seule. Pas besoin de toucher l'écran. Très utile quand il conduit.

10 Le profil et les papiers

Le chauffeur peut prendre en photo sa **carte pro**, son **assurance**, ses **conditions générales**, et les envoyer dans l'appli. Il peut aussi choisir ses départements préférés et bloquer des collègues qu'il ne veut pas voir.

11 On rend l'appli jolie ✨

Animations qui glissent, transitions douces, mode sombre, **icône de l'appli, écran de démarrage** avec le logo TaxiLink en jaune. C'est l'étape « maquillage » : tout marche déjà, mais on la rend belle.

12 On la donne à 10 vrais chauffeurs pour tester 🚗

On envoie l'appli à 10 chauffeurs amis. Ils l'utilisent pendant **1 semaine** en vrai sur la route. On regarde s'il y a des bugs, des trucs qu'ils n'aiment pas. On corrige.

🏁 Fin de Phase 1

La voiture du chauffeur est prête. C'est l'app la plus importante, et celle qui prend le plus de temps. Après elle, les autres iront plus vite.

4. Phase 2 — La voiture du patron (5 étapes) 🧑‍💼

Le patron travaille surtout sur son ordinateur, donc son appli mobile fait juste les **trucs urgents** :

1 Démarrage

On crée l'appli, on branche la connexion (comme étapes 1+2 du chauffeur).

2 Page d'accueil

Il voit tout de suite combien de courses du jour, alertes du genre « un chauffeur a refusé une course ».

3 Liste des courses

Il peut voir, filtrer, et **republier** une course dans un autre groupe.

4 Agenda

Il voit le planning de ses chauffeurs (lecture seule, pour modifier il prend son ordi).

5 Notifications urgentes

« Ding ! » quand une course n'est pas prise depuis 5 minutes, ou qu'un chauffeur s'est déconnecté pendant une course.

5. Phase 3 — La voiture du patient/famille (5 étapes) 🧓

Pour les personnes âgées qui prennent le taxi médical (et leur famille). Très simple, très gros boutons :

1 Connexion par SMS

Pas de mot de passe, on tape son numéro, on reçoit un code par SMS. Plus simple pour les seniors.

2 Liste des courses passées et à venir

« Votre prochain rendez-vous, jeudi 14h, hôpital de la Timone ».

3 Suivre le chauffeur en direct 📍

La mamie voit sur la carte la voiture du chauffeur qui approche, avec sa **photo, son nom et sa plaque**. Comme Uber.

4 Historique et factures

Elle peut télécharger ses factures en PDF.

5 Notifications

« Votre chauffeur est en route », « il est arrivé devant chez vous », « course terminée ».

6. Phase 4 — Mise dans les magasins d'apps (3 étapes)



Maintenant que les 3 voitures roulent, on les met dans les **2 grands magasins d'apps** : App Store (Apple) et Play Store (Google).

1 Préparer les photos et la pub

On prend des belles captures d'écran de chaque appli, on écrit la description, on fait peut-être une mini-vidéo.

2 Politique de confidentialité + envoi aux magasins

On remplit des papiers obligatoires (RGPD, ce qu'on collecte comme infos), on appuie sur « Soumettre ».

3 Apple et Google vérifient

Ils regardent l'appli pendant quelques jours, parfois ils disent « non, refais ça », on corrige, on renvoie. Quand c'est validé, **les apps sont disponibles publiquement** sur les téléphones du monde entier 🎉.

Partie 2 — Explication technique pédagogique

Mêmes étapes, mais avec le vocabulaire des développeurs (chaque terme expliqué)

7. La stack technique en 5 minutes

« **Stack technique** » signifie l'ensemble des technologies qu'on utilise pour construire l'appli. Voici la stack TaxiLink en quelques mots, chaque outil expliqué :

React Native + Expo

React Native est un framework qui permet d'écrire une appli mobile (iOS + Android) avec du code JavaScript / TypeScript. Le code est ensuite compilé en composants natifs (vrais boutons iOS, vrais boutons Android), *pas en webview*. **Expo** est une surcouche qui simplifie React Native : elle fournit des modules prêts à l'emploi (GPS, notifications, caméra...) et un service cloud appelé **EAS Build** qui compile l'appli sans avoir besoin d'un Mac pour iOS.

Next.js (apps/web)

Le site web actuel est en **Next.js 14** avec **App Router**. C'est un framework React qui ajoute le rendu côté serveur (SSR), le routing par fichiers, et l'optimisation automatique. Il continue à tourner en parallèle des apps mobiles, sans régression.

Monorepo (npm workspaces + Turbo)

Un **monorepo** est un seul dépôt Git qui contient plusieurs applications et plusieurs librairies internes. **npm workspaces** gère les dépendances communes ; **Turbo** orchestre les builds et les caches. Notre arborescence : **apps/web** (le site), **apps/mobile-driver** (à venir), **packages/*** (les librairies partagées).

Supabase

Supabase est notre backend : base de données **PostgreSQL**, authentification, stockage de fichiers, et **realtime** (notifications de changements en direct). Tout le code TaxiLink (web et mobile) parle au même Supabase.

NativeWind v4

NativeWind est l'équivalent de Tailwind CSS pour React Native. On écrit des classes type `bg-primary p-4 rounded-3x1` directement dans le JSX, et elles sont compilées en styles natifs. Cohérence parfaite avec le web qui utilise déjà Tailwind.

Sentry

Sentry capture les erreurs et crashes en production. Le web utilise `@sentry/nextjs` ; le mobile utilisera `@sentry/react-native`. Même DSN, même dashboard, source maps automatiques.

8. Phase 0 technique — Refactor monorepo (substantielle done)

Avant de bootstrapper l'appli mobile chauffeur, on a extrait du code de `apps/web` vers des **packages partagés cross-platform**. Objectif : éviter que mobile-driver ne duplique 60-70 % du code déjà écrit côté web.

5 packages créés ou étendus

Package	Contenu
<code>@taxilink/design-tokens</code>	couleurs, ombres, rayons, polices, animations. Importé par <code>tailwind.config.ts</code> (web) et le NativeWind preset (mobile).
<code>@taxilink/supabase-types</code>	types TypeScript générés depuis le schéma Supabase (Database, Mission, Driver...). Source unique web + mobile.
<code>@taxilink/core</code>	types métier (Mission, Driver, Group), mocks pour tests, validators (auth/missions), utilitaires (départements, polyline, géolocalisation).
<code>@taxilink/services</code>	22 services cross-platform : authService, missionService, paymentService, voiceParseService, etc. Tous consomment Supabase via injection de dépendance (DI).
<code>@taxilink/stores</code>	11 stores Zustand : driverStore, missionStore, gpsStore, etc. Persistence opt-in via <code>@react-native-async-storage</code> côté mobile, <code>localStorage</code> côté web.

Pattern critique : injection de dépendance (DI)

Le client Supabase n'est plus créé par les services eux-mêmes. À la place, le package expose `getSupabaseClient()` / `setSupabaseClient(client)`. Au démarrage de chaque appli :

- **apps/web** appelle `setSupabaseClient(createBrowserClient(...))`
- **apps/mobile-driver** appellera `setSupabaseClient(createClient(URL, ANON, { auth: { storage: SecureStore } })))`

Résultat : un même service `authService.signIn()` fonctionne dans les deux environnements, parce qu'il appelle juste `getSupabaseClient()` pour récupérer le client courant.

Pattern bridge lazy via Proxy

`apps/web/src/services/_bridge.ts` expose `bridgeService(s)` qui retourne un **Proxy**. Au premier accès à une méthode, le Proxy déclenche `ensureBridge()` qui appelle `setSupabaseClient + setApiImpl + setBroadcastImpl + setErrorReporter` une seule fois (idempotent). Cela évite les erreurs **TDZ (Temporal Dead Zone)** en tests Vitest, où les `vi.mock` référencent des constantes déclarées plus bas.

✅ État final Phase 0

- **1127 / 1127 tests passent**
- Type-check vert sur tous les workspaces
- Web toujours en production sans régression
- Mobile-driver peut consommer `@taxilink/services` et `@taxilink/stores` tels quels
- **Temps réel : 1 jour** (vs 1 semaine prévu dans `MOBILE_STRATEGY.md`)

9. Phase 1 technique — App chauffeur, semaine par semaine

Semaine 1 — Bootstrap

`npx create-expo-app apps/mobile-driver --template tabs` puis conversion en **Expo Router v3** (routing par fichiers, mental model identique à Next.js App Router). Setup TypeScript strict, ESLint, Prettier alignés sur le web. Ajout de `@taxilink/design-tokens` via NativeWind preset. Branchement **Sentry React Native**. Premier build EAS Development → APK installable sur device Android, IPA via TestFlight pour iOS.

Critère de sortie : l'appli boot sur device, affiche « Hello TaxiLink », crash test Sentry visible dans le dashboard.

Semaine 2 — Authentification

Écrans login (email + password), signup, OAuth Google (`expo-auth-session`), complete-profile, mot de passe oublié. Storage du refresh token Supabase via **expo-secure-store** (Keychain iOS / Keystore Android). **Persistence « type Instagram »** : tokens longue durée, déconnexion uniquement via bouton explicite. Réutilisation directe des services `authService` et `profileService` du package.

Semaine 3 — Liste missions + realtime

Écran principal `app/(driver)/missions.tsx` avec **FlatList virtualisée** pour gérer 100+ missions sans lag. Filtres `dept_preferences` côté serveur (`missionQueries.getAvailable`). Realtime via `useMissionRealtime` — abonnement Supabase real-time sur la table missions. **Patch-diff** (insertion / mise à jour / suppression ciblée), pas de full reload.

Semaine 4 — Détail mission + carte + accept/refuse

Écran `app/(driver)/missions/[id].tsx` avec **react-native-maps** (Apple Maps iOS / Google Maps Android natives, pas Mapbox). Marqueurs départ/arrivée, polyline OSRM. Bouton accepter → `missionService.acceptMission()` avec **optimistic update** (UI mise à jour avant la réponse serveur, rollback en cas d'erreur).

Semaine 5 — GPS foreground + heartbeat

`useDriverPositionPush` adapté pour `expo-location.watchPositionAsync` (precision 10m, interval 5s). Toggle « En ligne / Hors ligne » persistant via Zustand + secure-store.

`useDriverHeartbeat` réutilisé tel quel (timestamp envoyé toutes les 30s pour confirmer que l'appli est vivante).

Semaine 6 — GPS background (la phase la plus risquée)

expo-task-manager : `defineTask('GPS_TASK', ...)` qui pousse les positions vers Supabase même quand l'appli est en arrière-plan ou écran éteint. `expo-location.startLocationUpdatesAsync` avec `accuracy: BestForNavigation`. **Permissions iOS** : `NSLocationAlwaysAndWhenInUseUsageDescription` dans `app.json` avec justification claire en français pour la review Apple. **Android** : `FOREGROUND_SERVICE_LOCATION` + notification persistante. **Battery optimization Android** : whitelist via `expo-battery` + tutoriel in-app.

⚠ Risque principal Phase 1

Apple peut rejeter la permission Always-Location si la justification est floue. Mitigation : justification claire + screen-record démo + demo account avec scénario CPAM dans le formulaire de soumission.

Semaine 7 — Push notifications

expo-notifications : token de chaque device stocké dans table `device_push_tokens`. **Edge Function Supabase** déclenchée sur INSERT mission : filtre chauffeurs en ligne du département, filtre groupes autorisés, filtre blocages, POST batch sur `https://exp.host/-/api/v2/push/send`. Notifications interactives (actions Voir / Ignorer). **Deep-link** tap → `/missions/[id]`.

Semaine 8 — Mode course actif

`useAutoMissionProgress` (réutilisable direct, pas de DOM). `useCourseGeofence` détecte arrivée par calcul de distance Haversine. Écran « course en cours » full-screen carte + ETA temps réel. Bouton SOS. **expo-keep-awake** activé pendant la course (écran reste allumé). Mode nuit : couleurs sombres sur la map, optionnel `expo-brightness`.

Semaine 9 — Voix mains-libres

Bouton volume hardware via **react-native-volume-manager**. Bouton écran flottant gros (FAB). **expo-audio** enregistre pendant maintien. Upload vers `/api/missions/parse-voice` (endpoint Whisper existant). Si réponse incomplète → `/api/missions/parse-voice-answer` (suite). Feedback haptique via **expo-haptics**. Test critique : enregistrement Bluetooth HFP mono 8 kHz → Whisper gère bien.

Semaine 10 — Profil + documents + admin

Écrans profil, documents (carte pro, assurance, CG), préférences départements, blocking collègues. Réutilisation directe `documentService`, `driverService`, `userPrefsService`, `driverBlockService`. Picker via **expo-image-picker** + redimensionnement local. Upload Supabase Storage en streaming via `FileSystem.uploadAsync`.

Semaine 11 — Polish UI/UX

Animations **Reanimated 3** sur transitions d'écrans, swipe-to-accept missions, pull-to-refresh custom. Splash screen branded (logo wordmark Plus Jakarta Sans 800). Icône appli générée depuis SVG 1024×1024. Dark mode. Empty states / error states soignés. i18n préparée en clés `t('...')` même si on lance en français only.

Semaine 12 — Beta interne et externe

TestFlight (Apple, gratuit, jusqu'à 10k testeurs, review légère pour la 1ère soumission ~24-48h). **Play Internal Testing** (Google, groupe email, pas de review). **10 chauffeurs beta** sélectionnés (mix Android/iOS, mix Marseille/Paris). Hotline Slack ou Discord pour bug reports. Sentry actif, dashboard hebdomadaire (crashes, ANR Android, taux de delivery push).

10. Phase 2/3/4 technique

Phase 2 — App patron (5 semaines)

Le patron travaille majoritairement sur PC, donc l'appli mobile est une vue simplifiée orientée alertes et actions rapides.

Semaine	Livrable technique
Sem 1	Bootstrap <code>apps/mobile-patron</code> , partage des packages, écrans auth (réutilise <code>authService</code> + multi-tenancy déjà en place).
Sem 2	Onglet Vue : KPIs jour, courses en cours, alertes (employé refuse course, course non prise depuis 5 min).
Sem 3	Onglet Courses : liste filtrable, détail, action « republier dans autres groupes ».
Sem 4	Onglet Agenda : vue planning lecture seule (création reste sur PC car <code>PartagerMissionModal</code> trop complexe pour mobile : BAN + OSRM + voice + tarif CPAM).
Sem 5	Push notifs critiques + polish.

Phase 3 — App patient/famille (5 semaines)

Patients majoritairement seniors (transport CPAM). UX large polices, contraste fort, peu d'écrans, gros boutons. À tester avec utilisateurs réels seniors avant lancement.

Semaine	Livrable technique
Sem 1	Bootstrap <code>apps/mobile-patient</code> , auth simplifiée via SMS OTP (Supabase Phone Auth), pas de mot de passe.
Sem 2	Liste courses passées et à venir, filtres simples.
Sem 3	Suivi temps réel : map du chauffeur qui arrive, ETA actualisé, photo + nom + plaque + téléphone.
Sem 4	Historique + factures (PDF) + RGPD self-service.
Sem 5	Push notifs (« Chauffeur en route », « arrivé », « course terminée ») + polish.

Phase 4 — EAS Build et soumission stores (3 semaines)

EAS Build (Expo Application Services) compile dans le cloud les binaires iOS ([.ipa](#)) et Android ([.aab](#)) signés, puis **EAS Submit** les pousse automatiquement vers App Store Connect et Google Play Console.

Sem 1 — Assets stores

Screenshots iOS (5,5" 6,5" 6,7"), Android (téléphone + tablette), copywriting **ASO** (App Store Optimization) en 3 versions A/B, vidéo preview optionnelle. Politique de confidentialité dédiée mobile (taxilink.fr/legal/privacy-mobile).

Sem 2 — Soumission

EAS Submit configuré avec credentials **Apple Developer** (99 €/an) et **Google Play Console** (25 € one-shot). Pour Apple : compte individuel recommandé (pas de D-U-N-S, vs 5-14j ouvrés). Pour Google : **20 testeurs internes pendant 14 jours** obligatoires depuis nov. 2023 — à anticiper en démarrant les testeurs dès la beta de la Phase 1.

Sem 3 — Itérations review

Apple : 24-48h pour la 1ère review en moyenne, 1-3 cycles de retours fréquents (permissions floues, screenshots non conformes, etc.). Google : plus permissif sur la 1ère review. **Publication progressive** (10 % → 50 % → 100 %) pour limiter le blast radius en cas de bug.

11. Récap d'avancement

Phase	Estimation prudente	Réel	État
Phase 0 — Refactor monorepo	1 semaine	1 jour (2026-05-10)	✅ Done
Phase 1 — App chauffeur	16 semaines	—	🕒 À démarrer
Phase 2 — App patron	6 semaines	—	🕒 Après Phase 1
Phase 3 — App patient/famille	6 semaines	—	🕒 Après Phase 2
Phase 4 — Polish + stores	4 semaines	—	🕒 Tout à la fin

Avancement global de l'écosystème TaxiLink

- **Web (apps/web)** — en production, ~85-90 % du périmètre prévu (dashboard chauffeur, dashboard patron Phase 1, marketplace, GPS auto-progress, voice Whisper, blocages, dispatch ronds élargissants, auth persistante, doc CPAM...)
- **Mobile (3 apps natives)** — ~3 % (foundation Phase 0 livrée, mobile-driver peut consommer les packages directement)
- **Avancement combiné estimé pondéré par effort : ~38 % du livrable final**

🎯 Prochaine étape immédiate

Phase 1 — Semaine 1 : bootstrap `apps/mobile-driver`. Pure infrastructure, pas de UI à designer encore. Une fois bootté, on discute le design écran par écran (PWA actuelle vs mockups Claude vs autres options) avant d'attaquer Semaine 2.

 En parallèle du code, à lancer dès maintenant

- Création des comptes **Apple Developer** (99 €/an) et **Google Play Console** (25 € one-shot)
- Recrutement de **20 testeurs internes Play** (compteur de 14 jours obligatoire)
- Identification des **10 chauffeurs beta** de la Semaine 12